3.0 Fundamentals of Software Testing

Why is testing necessary?

Why Testing is Necessary

Software Systems – Some context

- Software Systems are now part of our everyday life
- They are used almost everywhere, for example in:
  - Banking and Financial institutions
  - Retail industry
  - Central and Local Government
  - Transport (e.g. Planes, Trains and Automobiles)
  - Medicine (Hospitals, research centres)
  - Home Entertainment
- We have all experienced Software Systems failing!

Why Testing is Necessary

Software Systems – When things go wrong

Software System Failures can lead to:

- Human Injury or Death
  - e.g. airplanes crashing
- Technological disasters
  - e.g. Missile Systems malfunctioning
- Legal action and associated costs
  - e.g. failure to meet contractual obligations
- Loss of reputations to suppliers and/or their customers;
  - e.g. mis-spelling company name on mail shots

Why do defects occur in software?

- Software is written by human beings
  - who know something, but not everything
  - who have skills, but aren't perfect
  - who do make mistakes (errors)
- Under increasing pressure to deliver to strict deadlines
  - no time to check but assumptions may be wrong
  - systems may be incomplete

Why Testing is Necessary

Causes of Software Failure

- A Human can make an **Error (**aka a **Mistake)**
- An **Error is** '*A Human Action that produces an Incorrect Result*'
- The Error can cause a **Defect** (aka a **Fault** or **Bug**) 
- A **Defect** is '*A flaw in a component or system that can cause the component or system to fail to perform its required function*'
- A **Defect** can be in the Software, System or in a Document

* ISTQB Standard Glossary of Terms

Recap



manifest



causes



Error
A human action that produces an incorrect result

Fault/Defect/Bug
A flaw in a component or system that can cause the component or system to fail to perform its required function

Failure
Deviation of the component or system from its expected delivery, service or result

Activity: Yes, defects occur because human beings are fallible. Discuss other causes that contribute to defects occurrence.

Failures can be caused by environmental conditions as well
- E.g. radiation, magnetism, electronic fields
- Pollution can cause faults in firmware or influence the execution of software by changing hardware conditions.

innovative ● entrepreneurial ● global www.utm.my

Why Testing is Necessary

The Role of Testing

Very strict, detail & systematic

- Rigorous testing of systems and documentation can:
  - reduce the risk of problems occurring in an operational environment
  - contribute to the quality of the software system
- How? By finding and correcting defects before the system is released for operational use
- Software testing may also be required to meet contractual or legal requirements, or industry-specific standards

innovative ● entrepreneurial ● global www.utm.my

Why Testing is Necessary

Testing and Quality

- Testing can help us measure the **Quality** of software
- Quality - '*The degree to which a component, system or process meets specified requirements and/or user/customer needs and expectations*'
- This is measured in terms of defects found
- Defects covering:
  - functional software requirements and characteristics
  - and non-functional software requirements and characteristics (e.g. reliability, usability, efficiency, portability and maintainability)
- Testing can give confidence in the **Quality** of the software if it finds few or no defects

Why Testing is Necessary

Testing and Quality

- A properly designed test that passes, reduces the overall level of **Risk** in a system
- **Risk** – '*A factor that could result in future negative consequences; usually expressed as impact and likelihood*'
- When testing does find defects, the **Quality** of the software system increases when those defects are fixed
- The **Quality** of systems can be improved through Lessons learned from previous projects
- Analysis of root causes of defects found in other projects can lead to Process Improvement
- Process Improvement can prevent those defects reoccurring
- Which in turn, can improve the **Quality** of future systems
- Testing should be integrated as one of the **Quality** assurance activities

Why Testing is Necessary

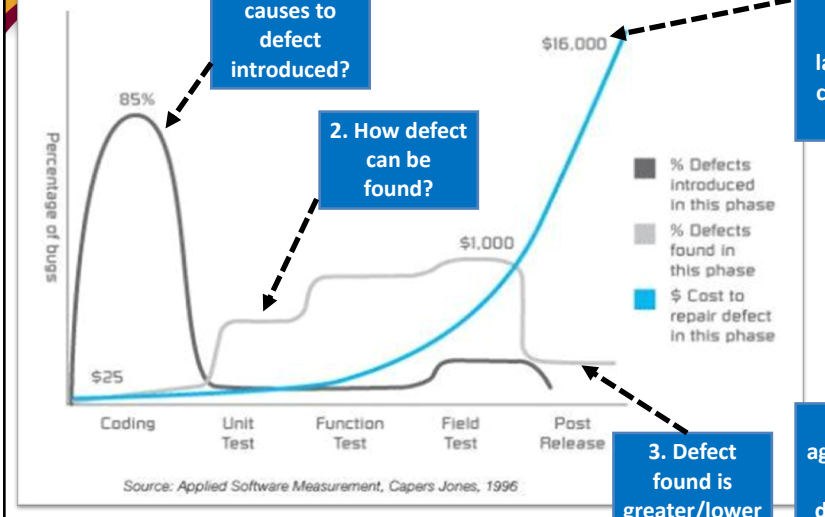
1. What causes to defect introduced?

2. How defect can be found?

3. Defect found is greater/lower after deployment?

4. Do you agree, finding and fixing defects able to increase the quality?

5. Fixing defects at latter phase cost greater budget.



Testing wisdom



By **Bill Hetzel 1988**

"Quality is not intangible.
The purpose of testing is to make quality visible.
Testing is the measurement of software quality!"



Why Testing is Necessary

Why exhaustive testing is impossible

- Exhaustive testing of complex software applications:
  - requires enormous resources
  - is too expensive
  - takes too long
- It is therefore impractical
- Need an alternative that is pragmatic, affordable, timely and provides results

Testing wisdom

By **Bill Hetzel 1988**

"In any form of testing it is impossible to achieve total confidence. The only exhaustive testing there is ……is so much testing, that the tester is exhausted!"



Why Testing is Necessary

Why don't we test everything ?

System has 20 screens
Average 4 menus / screen
Average 3 options / menu
Average of 10 fields / screen
2 types of input per field
Around 100 possible values

Approximate total for exhaustive testing:
20 x 4 x 3 x 10 x 2 x 100 = ? Tests

How long to run?

Test length = 1 sec then test duration = 17.7 days
Test length = 10 sec then test duration = 34 weeks
Test length = 1 min then test duration = 4 years
Test length = 10 mins then test duration = 40 years!

Why Testing is Necessary

So, How Much Testing is Enough?

- Deciding how much testing is enough should take account of:
  - the level of Risk
  - project constraints such as time and budget
- Risks should be evaluated at the Business Level, Technological Level, Project Level and Testing Level
- Risks are also used to decide where to start testing and where more testing is needed
- Risk considerations can include:
  - financial implication of software being released that is un-tested (support costs / possible legal action)
  - software being delivered late to market
  - potential loss of Life (safety critical systems)
  - potential loss of face (may have financial implications as well)









Why Testing is Necessary

So, How Much Testing is Enough?

- Risk analysis should be used to determine what to test in each component and just as importantly what not to test
- For example, an unacceptable risk would say we must test, an acceptable one perhaps not to test
- Testing is a risk-control activity that provides feedback to the stakeholders
- With this feedback the stakeholders can make informed decisions about the release of the software (or system) being tested

Why Testing is Necessary

So, How Much Testing is Enough?

- **Exit criteria** is used to determine when testing at any stage is complete

  The set of generic and specific conditions, agreed upon with the stakeholders, for permitting a process to be officially completed



- Exit criteria may be defined in terms of :-
  - Thoroughness – i.e. coverage or requirements
  - cost or time constraints
  - percentage of tests run without incident
  - number of faults remaining

Why Testing is Necessary

Example of exit criteria by phases;



What is testing?

What is testing?

A Definition (and a Misconception)

- When asked, people often think that Testing only consists of running tests, i.e. executing the software
- **Testing** – *'The process consisting of all life cycle activities, both static and dynamic, concerned with planning, preparation and evaluation of software products and related work products to determine that they satisfy specified requirements, to demonstrate that they are **fit for purpose** and **to detect defects**.'*
- Test execution is only a part of testing, but not all of the testing activities
- Test activities exist before and after test execution

What is testing?

A Definition (and a Misconception)

- Activities such as:
  - Planning and control
  - Choosing test conditions
  - Designing test cases
  - Checking results
  - Evaluating completion criteria
  - Reporting on the testing process and system under test
  - Finalizing or closure (e.g. after a test phase has been completed)
- Testing also includes reviewing of documents (including source code) and static analysis

What is testing?

Test Objectives

- There are different test objectives:
  - To find defects
  - To gain confidence about the level of quality and to provide information
  - To prevent defects
- Both dynamic testing and static testing can be used as a means for achieving these objectives
- They provide information in order to improve:
  - The system to be tested
  - The development and testing processes
  - Live operations

What is testing?

Test Objectives

- By designing tests early in the project life cycle it can help to prevent defects from being introduced into code
- Reviews of documents throughout the lifecycle (e.g. requirements and design) also help to prevent defects appearing in the code. More about this when we cover Static techniques

What is testing?

Test Objectives

- The Objectives of testing can vary depending on the stage of testing being conducted. E.g.:
  - Development testing (e.g. component, integration and system testing) - to cause as many failures as possible so that defects in the software are identified and can be fixed
  - Acceptance testing - to confirm that the system works as expected, to gain confidence that it has met the requirements
  - Maintenance testing - often includes testing that no new errors have been introduced during development of the changes
  - During Operational testing - may be to assess system characteristics such as reliability or availability

What is testing?

Test Objectives

- In some cases the main Objective of testing may be to assess the quality of the software (with no intention of fixing defects), to give information to stakeholders of the risk of releasing the system at a given time
- More on these test stages later in the course

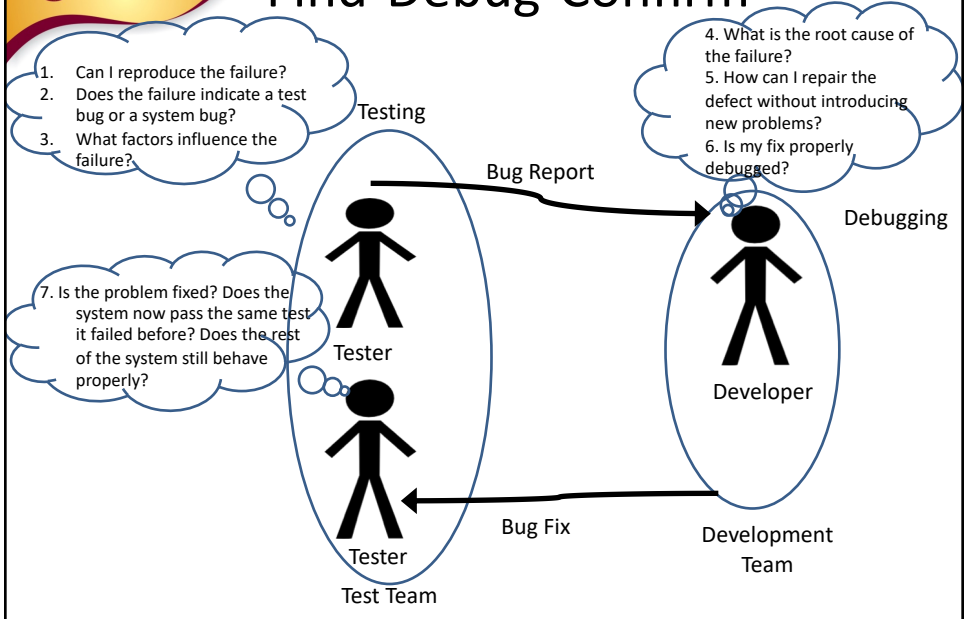
What is testing?

Testing v Debugging

- Debugging and testing are different:
  - Testing can show failures that are caused by defects
  - Debugging identifies the cause of a defect, repairs the code and checks that the defect has been fixed correctly
- Testing then ensures that the fix does indeed resolve the failure
- The responsibility for each activity is very different, i.e.
  - Testers test
  - Developers debug

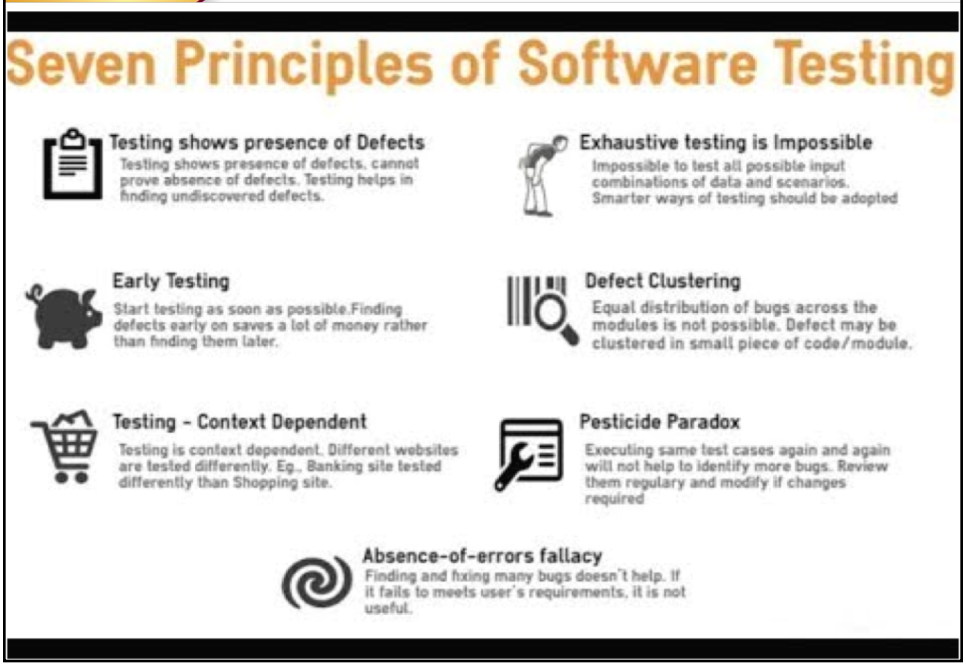


Find-Debug-Confirm



7 Testing Principles

General testing principles



General Testing Principles

The Seven Key Principles

1. Testing shows the presence of Defects

- We test to find Faults (a.k.a Defects)
- As we find more defects, the probability of undiscovered defects remaining in a system reduces.
- However Testing cannot prove that there are no defects present

General Testing Principles

The Seven Key Principles

2. Exhaustive Testing is Impossible!

- We have learned that we cannot test everything (i.e. all combinations of inputs and pre-conditions).
- That is we must Prioritise our testing effort using a Risk Based Approach.

General Testing Principles

The Seven Key Principles

3. Early testing

- Testing activities should start as early as possible in the development life cycle
- These activities should be focused on defined objectives – outlined in the Test Strategy
- Remember from our Definition of Testing, that Testing doesn't start once the code has been written!

General Testing Principles

The Seven Key Principles

4. Defect Clustering



- Defects are not evenly spread in a system
- They are 'clustered'



- In other words, most defects found during testing are usually confined to a small number of modules
- Similarly, most operational failures of a system are usually confined to a small number of modules



- An important consideration in test prioritisation!

General Testing Principles

The Seven Key Principles



5. The Pesticide Paradox

- Testing identifies bugs, and programmers respond to fix them
- As bugs are eliminated by the programmers, the software improves
- As software improves the effectiveness of previous tests erodes
- Therefore we must learn, create and use new tests based on new techniques to catch new bugs

- *N.B It's called the "pesticide paradox" after the agricultural phenomenon, where bugs such as the boll weevil build up tolerance to pesticides, leaving you with the choice of ever-more powerful pesticides followed by ever-more powerful bugs or an altogether different approach.' – Beizer 1995*

General Testing Principles

The Seven Key Principles

6. Testing is Context Dependent

- Testing is done differently in different contexts
- For example, safety-critical software is tested differently from an e-commerce site
- Whilst, Testing can be 50% of development costs, in NASA's Apollo program it was 80% testing
- 3 to 10 failures per thousand lines of code (KLOC) typical for commercial software
- 1 to 3 failures per KLOC typical for industrial software
- 0.01 failures per KLOC for NASA Shuttle code!
- Also different industries impose different testing standards

General Testing Principles

The Seven Key Principles

7. Absence of Errors Fallacy

- If we build a system and, in doing so, find and fix defects ....
  It doesn't make it a good system

- Even after defects have been resolved it may still be unusable and/or does not fulfil the users' needs and expectations